

ニューラルネットワークの最適化① 確率的勾配法

ニューラルネット > 最適化



NNの最適化に関する話を解説

- ニューラルネットワークは，構造の自由度などから有力なモデルの一つ
- しかし，パラメーターが複雑に絡み合うので，最適化は独自の工夫が凝らされる
 - 確率的勾配法
 - 誤差逆伝播
 - という基本の考え方2つを中心に解説

ニューラルネット＞最適化＞取扱いの難点①

データも大規模になりがち

- 複雑なデータに対応するために、NNのパラメーター数は大規模になる
 - パラメーターが多いので、訓練データも大規模にしないと性能が良くならない
- インターネット時代の到来により、大規模な訓練データが収集されるように
 - 2009年のImageNet：画像データが320万枚（その後1400万枚以上に拡張）
 - 2023年のRefinedWeb：テキストデータが数千億単語（数兆単語のデータのうち一部だけが公開）

【参考】 Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. 2009 IEEE Conference on Computer Vision and Pattern Recognition, 248–255. <https://doi.org/10.1109/CVPR.2009.5206848>

【参考】 Penedo, G., Malartic, Q., Hesslow, D., Cojocaru, R., Cappelli, A., Alobeidli, H., Pannier, B., Almazrouei, E., & Launay, J. (2023). The RefinedWeb dataset for Falcon LLM: Outperforming curated corpora with web data, and web data only (arXiv:2306.01116). arXiv. <https://doi.org/10.48550/arXiv.2306.01116>

ニューラルネット > 最適化



基本的には勾配法ベースの手法で最適化を行う

- 基本的に大域的最適解を求める解の公式はないので、勾配法で最適化する

- 復習) 勾配降下法のパラメーター更新式

$$\hat{\theta} \leftarrow \hat{\theta} - \eta \cdot \nabla L(\hat{\theta})$$

- ただし、ニューラルネットワークの性質に合わせて、経験則や理論解析に基づく様々な工夫が取り込まれている

- 単なる勾配降下法ではなく、かなり複雑な最適化手法が発達
- 最適化手法に取り入れる工夫も、帰納バイアスの一端を担う

ニューラルネット > 最適化 > 大規模化への対応

NNの学習に使うデータは大規模 → 訓練時の計算量を制御する工夫が必要

- 経験リスクに基づく目的関数

$$L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell((\mathbf{x}_i, y_i), g_{\boldsymbol{\theta}})$$

- この勾配は

$$\nabla L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \nabla \ell((\mathbf{x}_i, y_i), g_{\boldsymbol{\theta}})$$

- 例えば10億件のデータがあると、1回の更新だけでも10億件の勾配計算
 - あまりに多い場合、そもそもコンピューター1台には入りきらず実行不可能

ニューラルネット > 最適化 > 大規模化への対応

リスクのサンプル近似のために、必ずしも全てのデータを用いる必要はない

- ミニバッチ確率的最適化 (mini-batch stochastic optimization method)
 - 毎ステップ、データの一部を **ミニバッチ (mini-batch)** として取得して学習
 - **バッチサイズ (batch size)** = ミニバッチ内のデータ数 $|B|$ は1件～数百件
 - ミニバッチ B 上での訓練損失を小さくするように学習

$$\tilde{L}(\theta) = \frac{1}{|B|} \sum_{(x_i, y_i) \in B} \ell((x_i, y_i), g_{\theta})$$

- 通常のミニバッチの作り方
 - 訓練データをシャッフルし、バッチサイズごとに分割しておく

【用語】 ミニバッチではなく全部の訓練用データを一度に使う、と言いたい場合には「フルバッチ学習」や「バッチ学習」ということがある。

ニューラルネット > 最適化 > ミニバッチ確率的勾配法

ミニバッチ確率的勾配降下法 (mini-batch stochastic gradient descent)

- パラメーター θ を適当に初期化 ($\hat{\theta} \leftarrow \theta$)
- 以下を必要なだけ繰り返す (1周を**エポック**と呼ぶ)
 - 以下をすべてのミニバッチ B について実行する
 - ▶ ミニバッチ B で作った目的関数 $\tilde{L}$ の、現在の $\hat{\theta}$ における勾配 $\nabla \tilde{L}(\hat{\theta})$ を計算
 - ▶ 勾配の反対方向にパラメーターを少し更新 (学習率 $\eta > 0$)

$$\hat{\theta} \leftarrow \hat{\theta} - \eta \nabla \tilde{L}(\hat{\theta})$$

- 早期停止条件を満たしていたら中断
- 最終的な $\hat{\theta}$ を使用する

【用語】 エポック (epoch) は、同じ訓練データを何回学習に使ったかを数えたものとも言える。1エポック内では、各データ点はちょうど1回学習に使われる。

ニューラルネット > 最適化 > ミニバッチ確率的勾配法

通常の勾配法に加えて，次のような項目を決める必要がある

- パラメーター θ を適当に初期化 ($\hat{\theta} \leftarrow \theta$)
 - 何エポック学習するか？
- 以下を必要なだけ繰り返す（1周を**エポック**と呼ぶ）
 - データをシャッフルし，ミニバッチ $B_1, B_2, \dots$ に分割
 - バッチサイズ $|B|$ は？
 - エポック毎にシャッフルとミニバッチ再分割をするか？
 - 以下をすべてのミニバッチ $B_1, B_2, \dots$ について実行する
 - ▶ ミニバッチ B で作った目的関数 $\tilde{L}$ の，現在の $\hat{\theta}$ における勾配 $\nabla \tilde{L}(\hat{\theta})$ を計算
 - ▶ 勾配の反対方向にパラメーターを少し更新 $\hat{\theta} \leftarrow \hat{\theta} - \eta \nabla \tilde{L}(\hat{\theta})$ （学習率 $\eta > 0$ ）
 - 早期停止条件を満たしていたら中断
- 最終的な $\hat{\theta}$ を使用する

【用語】エポック（epoch）は，同じ訓練データを何回学習に使ったかを数えたものとも言える．1エポック内では，各データ点はちょうど1回学習に使われる．

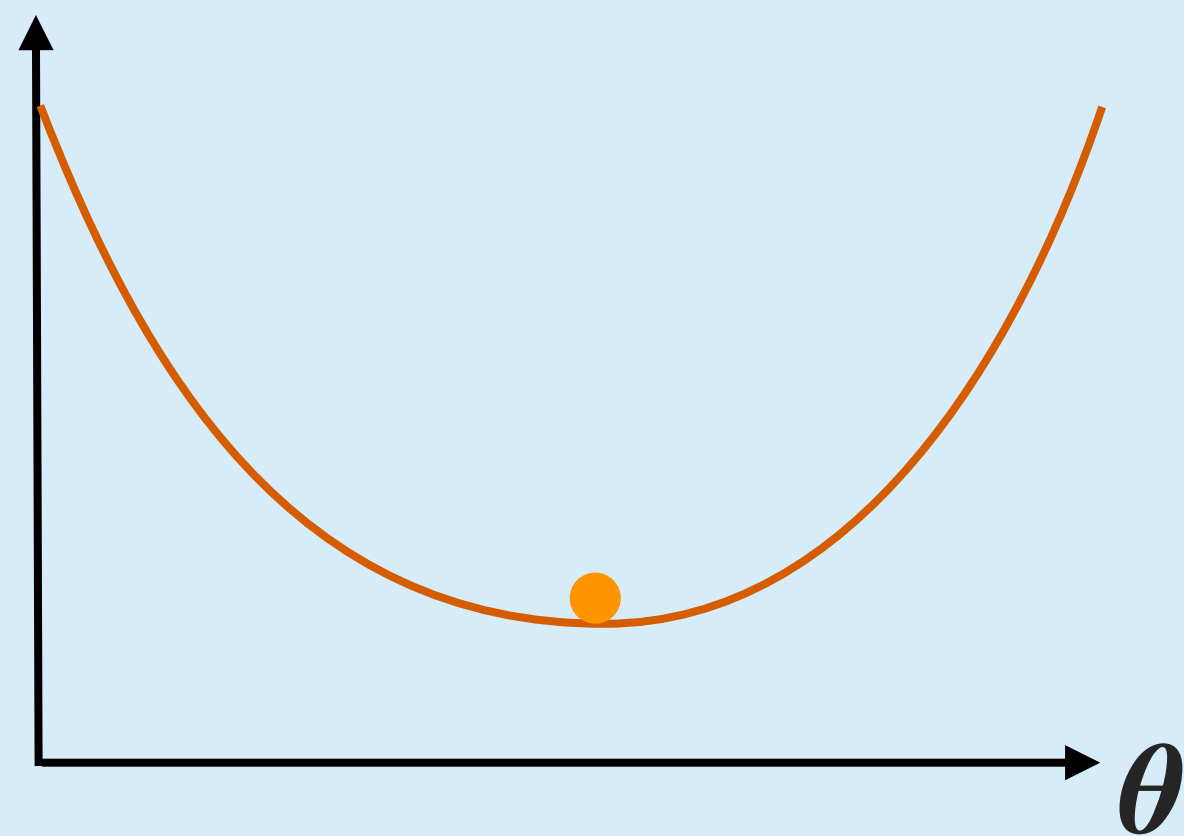
ニューラルネット＞最適化＞取扱いの難点②

目的関数の「地形」が複雑で、単純に坂を下るだけでは駄目かもしれない

線型モデルの最小二乗法など

凸最適化問題

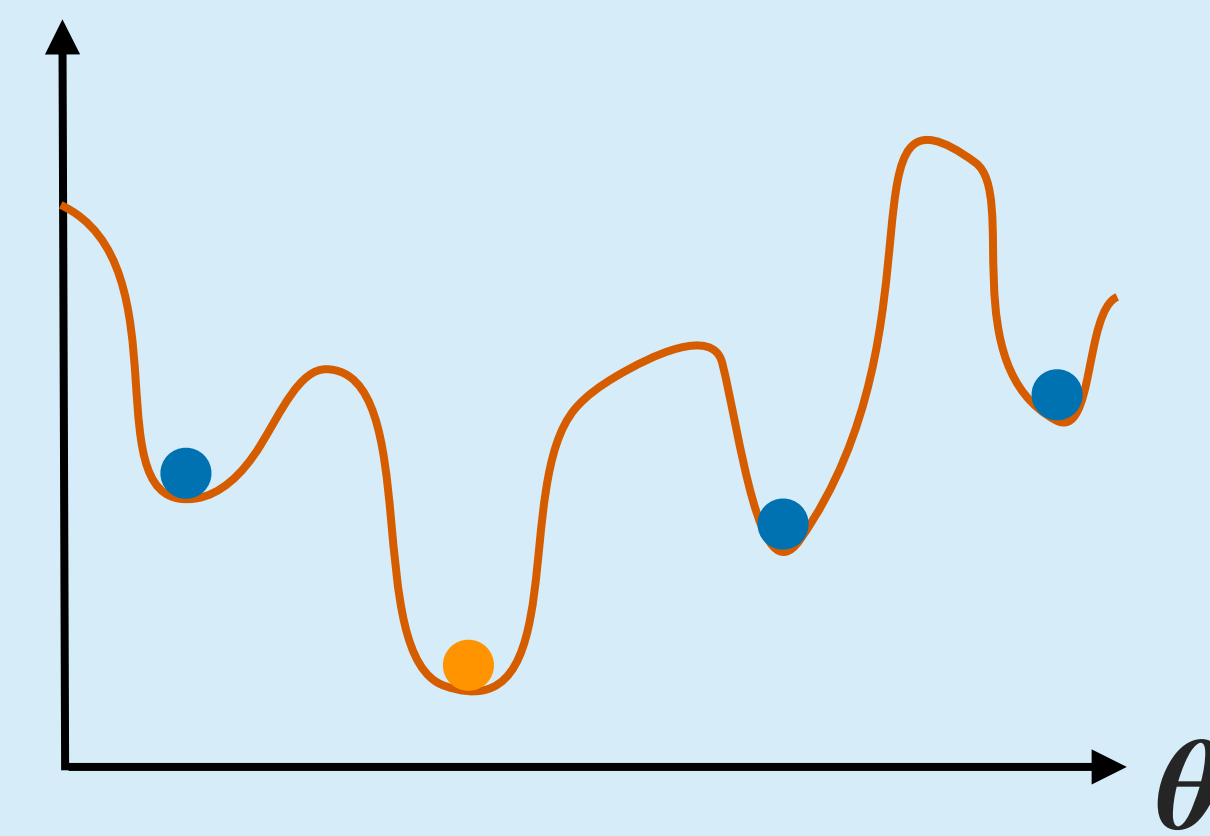
（大雑把に言って）解は一つで、
見つけやすい



ニューラルネットなど

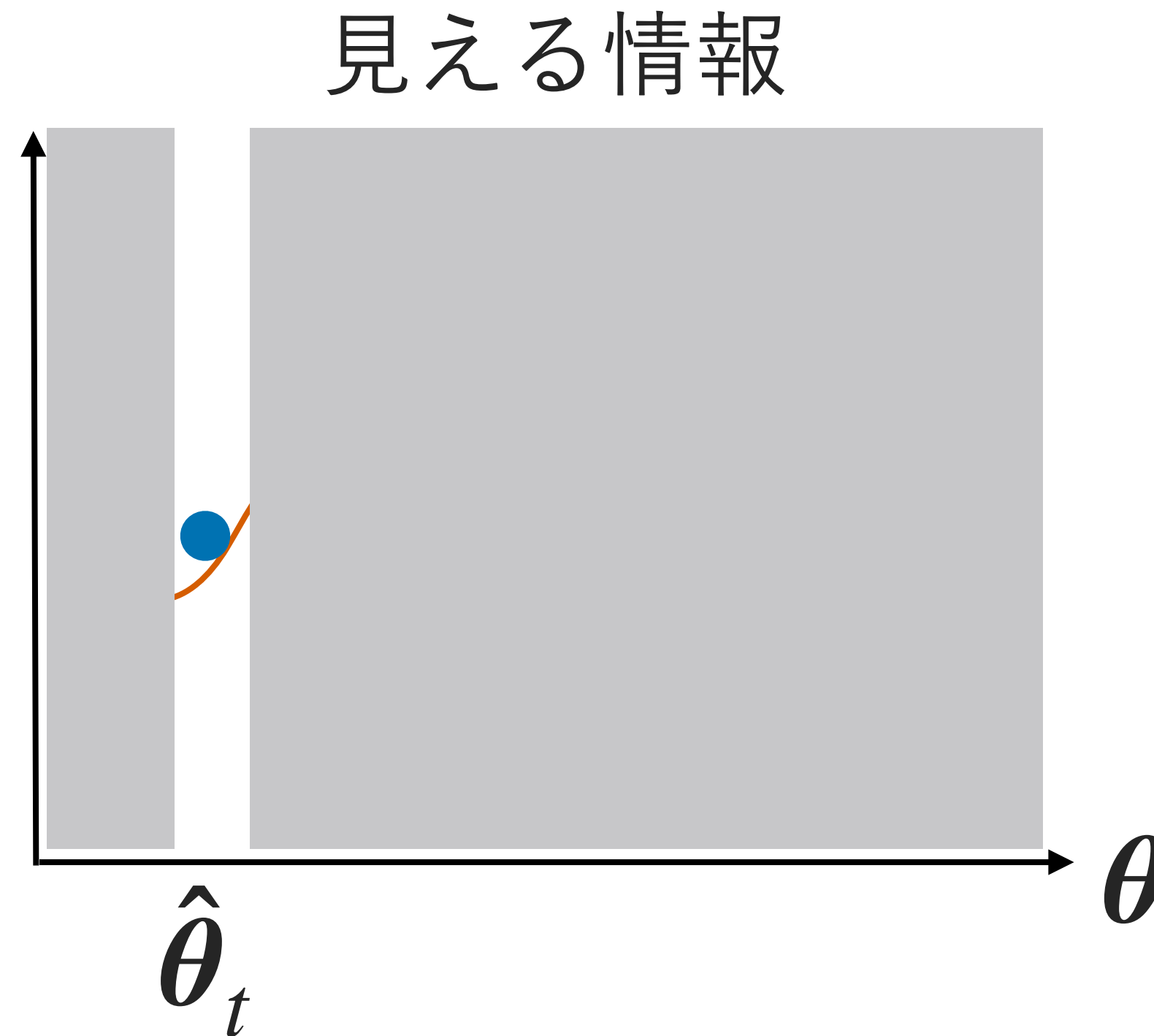
非凸最適化問題

局所的最適解が多数あり、大域的
最適解を求めるのが困難



局所的最適化手法

勾配法では基本的に「今いる場所の情報」だけでパラメーターを更新



$$\nabla L(\hat{\theta}_t) < 0$$

現在地での勾配情報だけを頼りに
パラメーターを更新

【補足】全体像は、あらゆるパラメーター値を試してその時の損失を記録していけば把握可能だが、高次元のパラメーターでそれは現実的には実行不可能。

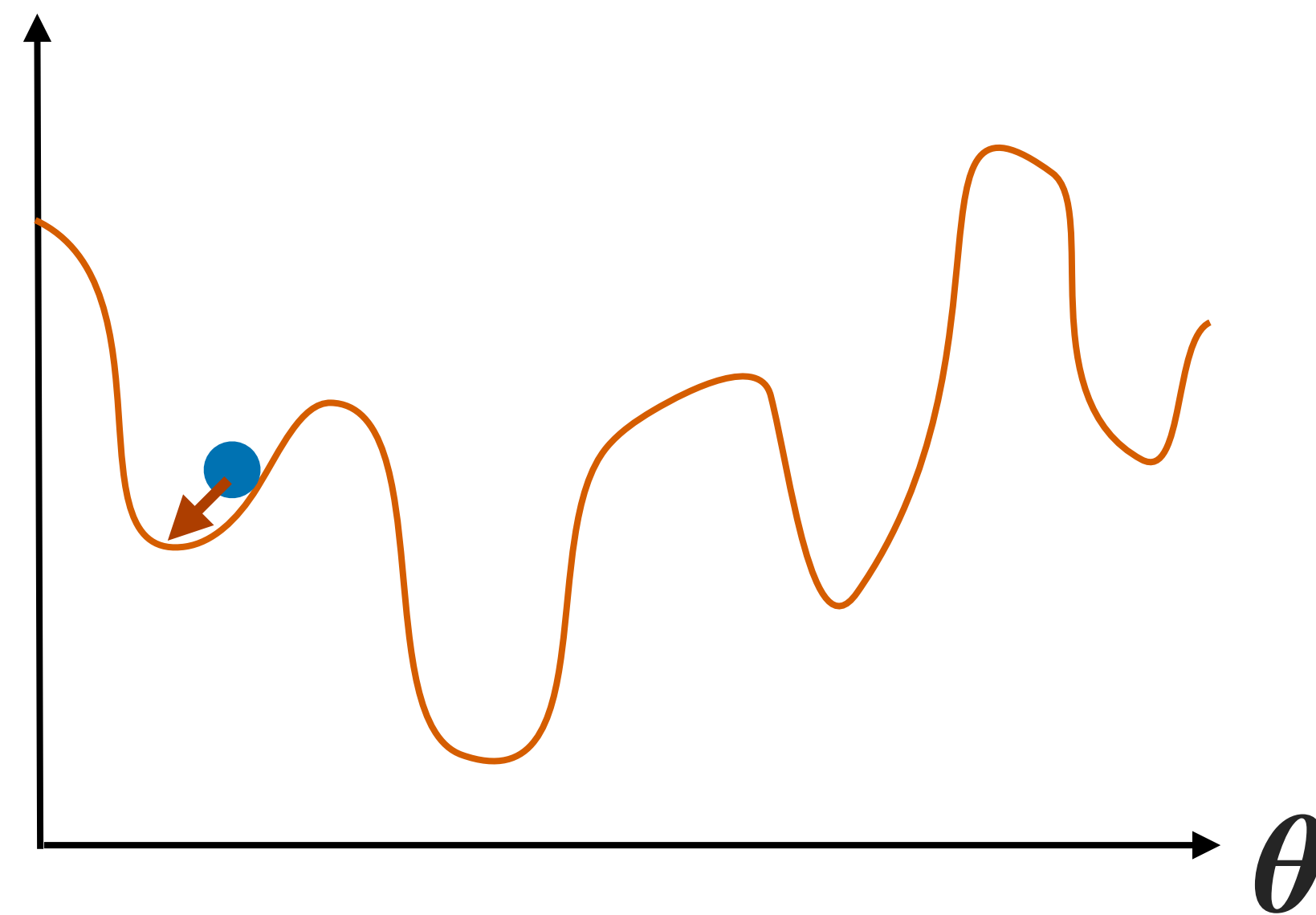
【補足】1次元だと、このように一旦上がって下がる必要があるなど、一段と難しい。高次元では逆に経路があって最適化しやすいこともある。

【補足】モメンタム法と呼ばれる工夫を取り入れた勾配法などでは、現在地だけでなく、それまでの勾配情報の履歴も加味して更新する。

局所的最適化手法

そのため、大域的な地形を踏まえた移動は基本的に難しい

全体像（見えない）



$$\nabla L(\hat{\theta}_t) < 0$$

←

損失値が低い θ に移動したいが、
左に動いてしまった

ニューラルネット > 最適化 > 複雑な構造を想定した工夫

実際は単純な勾配法ではなく，色々な工夫を組み込んだ最適化法が使われる

- 学習率の調整や，慣性項（momentum）を更新則に組み込むなどの工夫
- AdamやMuonやCosine Annealing Scheduleなどの工夫がある（詳細は略）

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)

【参考】 Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. Proceedings of 3rd International Conference for Learning Representations. ICLR 2015, San Diego. 【用語】 Adamという名前は，適応的なモーメント推定（adaptive moment estimation）に由来。

【参考】 Defazio, A., Yang, X.A., Khaled, A., Mishchenko, K., Mehta, H., Cutkosky, A., 2024. The road less scheduled. Presented at the The Thirty-eighth Annual Conference on Neural Information Processing Systems.

ニューラルネット＞最適化＞まとめ



ニューラルネットワークの最適化は、複雑で様々な工夫が必要

■ 主な複雑さ

- データ量が大规模
- 目的関数の”地形”が複雑（局所的最適解が存在するなど）

■ 主な解決方針

- 確率的勾配法により一度に全データを扱わなくてよい
- 局所的最適解を避けるための様々な工夫を凝らした最適化手法が開発（実用上はその時点で評判が良いものを選ぶのがよい）

【補足】線型モデルと異なり局所的最適解が存在するなどの違いがあることは確かだが、近年では、パラメーターを大量に持つニューラルネットワークの目的関数は逆に良い性質を持つのではないかと、などの観点から研究が進んでいる（そうでないと、実用上、高い性能を発揮するモデルが実際に得られている＝最適化が実践的には上手くできていることの説明がつかない、という面がある）。

ロードマップ

